

为了解决这个问题，本文提出了 *Adrenaline*，一种旨在提升 LLM 服务系统资源利用率和性能的注意力计算解耦与卸载机制。*Adrenaline* 的核心创新在于将解码阶段的部分注意力计算解耦并卸载到预填充实例上执行。解码阶段注意力计算固有的内存受限特性，使得这种卸载策略能够有效实施，并带来两个互补的优势：（1）提升了预填充实例的内存容量和带宽利用率；（2）增大了解码批次大小，从而提升了解码实例的计算利用率——两者共同提升了整体系统性能。*Adrenaline* 通过三项关键技术实现这些收益：低延迟解码同步、资源高效的预填充共置以及负载感知的卸载调度。实验结果表明，与最先进的系统相比，*Adrenaline* 使预填充实例获得了 2.28× 倍的内存容量提升和 2.07× 倍的内存带宽利用率提升，解码实例的计算利用率最高提升了 1.67× 倍，整体推理吞吐量提升了 1.68× 倍。

1 Introduction

随着先进的 Transformer 架构的发展和数十亿级参数规模的出现，大语言模型（LLMs）在内容生成方面取得了显著成功，其应用广泛存在于聊天机器人 [30]、写作助手 [26] 和代码生成 [18] 等领域。鉴于高质量的 LLM 推理通常需要高性能的 GPU 加速器，许多流行的 LLM，例如 GPT [8]、LLaMA [11]、Qwen [29] 和 Mistral [14]，都作为云服务提供，以便为推理查询提供快速且准确的响应。在这些 LLM 服务系统中，满足延迟的服务水平目标（SLOs）对于确保用户请求在严格的截止时间内得到处理至关重要。同时，服务提供商寻求通过优化 GPU 利用率和最大化吞吐量来降低服务成本，以处理尽可能多的请求。

在 LLM 服务系统中执行每个请求涉及两个连续的阶段。*prefill phase* 阶段并行计算所有提示词元以生成第一个输出词元。*decoding phase* 阶段随后根据先前的上下文（包括已计算的提示词和先前生成词元的中间结果，即 *KV cache*）顺序地预测每个模型前向步骤的一个输出词元。由于许多词元的并行处理，预填充阶段通常是计算密集型的，其延迟由首词元时间（TTFT）衡量。相比之下，解码阶段由于需要频繁加载不断增长的 KV 缓存而成为内存密集型，其延迟由每个输出词元时间（TPOT）衡量。由于预填充步骤通常比解码步骤产生高得多的延迟，在同一 GPU 上运行这两个阶段会导致显著的干扰，从而增加同一批次内预填充阶段的 TTFT 和解码阶段的 TPOT [32]。

为了减少预填充阶段和解码阶段之间的干扰，现代 LLM 服务系统通常采用预填充-解码（PD）解耦 [20, 22, 32]。通过将预填充和解码阶段分配给独立的 GPU，PD 解耦消除了两个阶段之间的干扰，使得每个阶段能够独立地满足其各自的服务水平目标（SLOs）。此外，将预填充和解码阶段解耦到不同的 GPU 池中，能够实现自动且灵活的资源扩展，以适应每个阶段不同的资源需求。如今，PD 解耦正成为生产级 LLM 服务系统的 *de-facto* 实践，例如 vLLM [16]、NVIDIA Dynamo [6]、Kimi 的 Mooncake [22] 和 Deepseek [17]。

然而，我们观察到，在 LLM 服务系统中进行 PD 解耦会导致 GPU 资源（无论是计算资源还是内存资源）的显著利用不足。具体来说，运行计算密集型的预填充阶段的 GPU 通常其 HBM 容量和带宽利用率较低。相反，内存密集型的解码阶段则面临计算资源利用率低的问题。如图 1a 所示，预填充实例的 HBM 带宽利用率低于 30%。此外，图 2 显示，在现有 PD 解耦系统（即 vLLM [16]）的预填充实例中，HBM 容量利用率低于 21%。同时，对于不同的批大小，解码实例的计算利用率低于 26%。考虑到 GPU 的高昂成本，如此低的资源利用率会导致推理成本增加。为了探究这种利用不足的原因，我们测量了预填充和解码阶段内部组件的资源消耗。我们的实验结果（§2.3）表明，预填充阶段中的四个主要组件（包括 QKV 投影、注意力计算、输出投影和前馈网络）都是计算密集型的。在解码阶段，图 2 显示，预热后（`gpu_memory_utilization` 为 0.8）HBM 容量利用率为 75.5%，而用于注意力计算的 KV 缓存占用了 57.3% 的 HBM 容量。此外，内存密集型的注意力核消耗了大量的 GPU 时间（例如，在图 3 中，当批大小为 80 时，每个 Transformer 层的执行时间中有 69.5% 用于注意力计算）。总的来说，

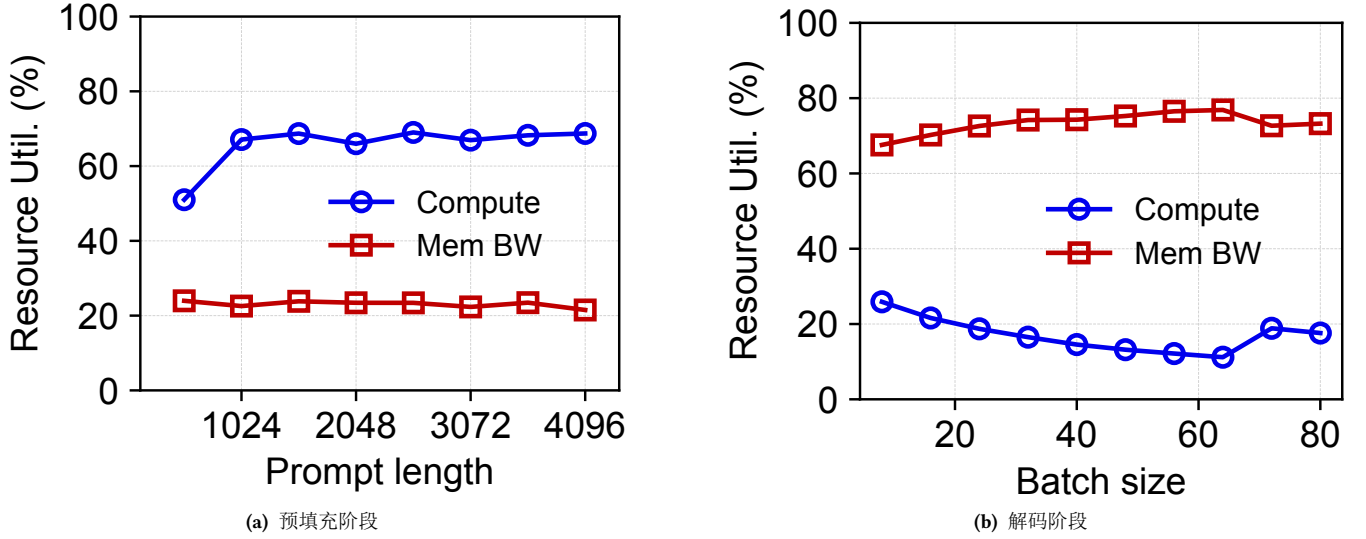


图 1. Llama-2 7B 在解耦的预填充和解码阶段的资源利用率。解码阶段的序列长度为 1K。

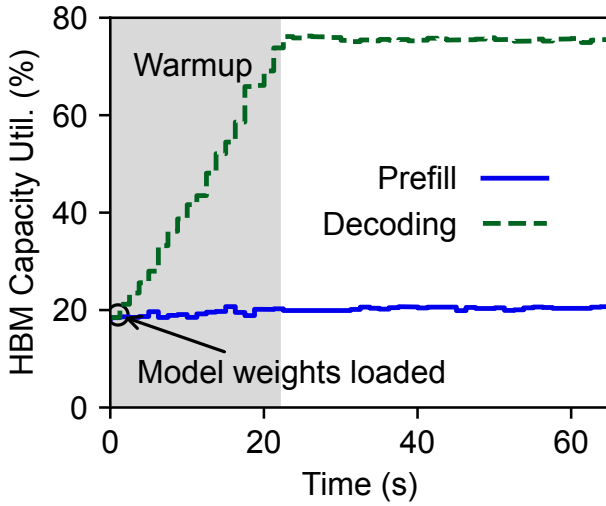


图 2. 服务 Llama-2 7B 时的 HBM 利用率。

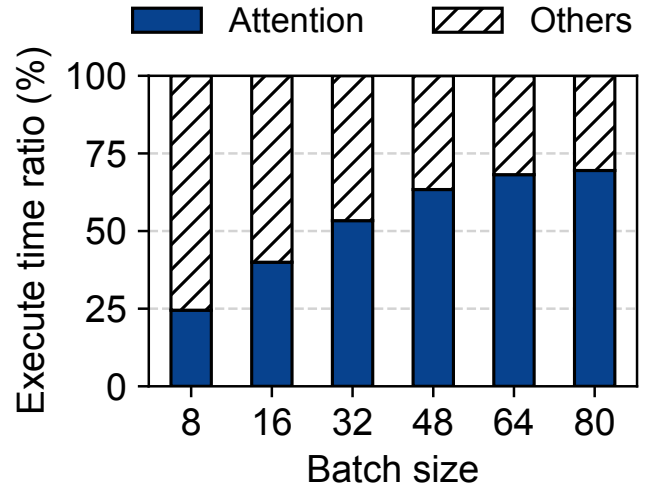


图 3. 解码注意力机制的执行时间。

解码实例中内存资源消耗高，主要是由于注意力计算所需的 KV 缓存的存储和访问 [12]。基于上述观察，本文提出 *Adrenaline*¹，一种用于具有 PD 解耦的 LLM 服务系统的 A 注意力 d 解 re 聚 n a 与 l 卸 in 载 e 机制，以有效提高 GPU 资源利用率，从而提升 LLM 推理性能。*Adrenaline* 将解码阶段的部分注意力计算解耦并卸载到运行预填充阶段的 GPU 上。这种方法通过将内存密集型的注意力计算任务从解码阶段卸载出去，提高了预填充实例中 HBM 容量和带宽的利用率。此外，它增加了解码实例中的总批次大小（本地批次大小与卸载批次大小之和），从而提高了其计算利用率。然而，实现注意力解耦与卸载面临着若干挑战。

1) Additional attention synchronization overheads in the decoding phase. 由于只有部分解码注意力任务被卸载，剩余的解码注意力任务仍在原始的解码阶段进行计算。因此，卸载的注意力必须在单个解码注意力步

¹肾上腺素是一种能迅速提升身体反应能力的激素。类似地，我们提出的技术如同注入 LLM 服务系统的肾上腺素，旨在提升推理吞吐量。

骤内（通常小于 1 ms）与解码计算的其余部分同步。此同步过程中的任何延迟都可能导致每个 Transformer 层出现 *attention stall*，从而增加 TPOT。

2) Performance interference in prefill instances. 卸载的解码注意力任务与预填充阶段共享 GPU 资源。例如，卸载的 KV 缓存会占用 HBM 容量，而执行注意力计算则会消耗计算资源和 HBM 带宽。这些任务的共存会导致对 GPU 资源的争用，这可能引起性能干扰并降低整体系统效率。

3) Efficient control of offloading rate. 将过多的注意力计算任务卸载到预填充实例可能导致 GPU 资源过度消耗，这很可能会增加 TTFT。反之，卸载过少的任务可能无法抵消同步开销。考虑到序列长度 [16] 的动态性以及预填充和解码实例中负载的波动，卸载策略需要具备灵活性。它应能高效地编排解耦后的 LLM 服务系统，以在实际工作负载下提升性能。为应对这些挑战，*Adrenaline* 采用了三项关键技术。首先，*Adrenaline* 通过缩短解码阶段与卸载注意力计算之间的同步关键路径，采用了一种低延迟同步策略。利用 CUDA 图技术来减少注意力卸载带来的额外内核启动开销。其次，为了减轻性能干扰，*Adrenaline* 探索了计算受限的注意力计算，并引入了一种资源高效的预填充共置机制来隔离 GPU 资源。最后，为了确定最优的卸载率，*Adrenaline* 利用了一种负载感知的卸载调度策略，该策略能动态调整卸载率。这一调整基于预填充和解码阶段实时的计算与内存资源利用率，以及推理服务等级目标。卸载率会持续更新以响应负载波动。

我们在 vLLM [16] 的基础上实现了 *Adrenaline*。实验结果表明，对于真实世界的工作负载（ShareGPT [5]），*Adrenaline* 的注意力卸载使预填充实例的内存利用率最高提升了 2.07×，解码实例的计算利用率最高提升了 1.67×。与 vLLM 相比，整体输出令牌吞吐量最高提升了 1.68×。*Adrenaline* 的源代码已在 Github 上发布²。总而言之，本文做出了以下贡献。

- 我们识别并分析了具有预填充-解码解耦的 LLM 服务系统中的 GPU 资源未充分利用问题，并探究了其根本原因。
- 我们提出了 *Adrenaline*，这是一种简单而有效的解决方案，通过注意力解耦与卸载来提升 LLM 服务系统中的资源利用率和吞吐量。
- 我们在 *Adrenaline* 中引入了三项关键技术，包括低延迟解码同步、资源高效的预填充共置以及负载感知的卸载调度，以实现高效的注意力卸载。
- 我们在 vLLM 上实现并评估了 *Adrenaline*，证明了其在提升性能和资源利用率方面的有效性。

2 Background and Motivation

2.1 Large Language Model Inference

Transformer Architecture. Transformer 网络 [27] 是 LLM 中的基础架构，已被许多流行模型 [11, 24, 25] 广泛采用。Transformer 由多个重复的 Transformer 层组成，每层包含四个主要计算步骤：(1) 查询 (q)、键 (k) 和值 (v) 的线性投影，统称为 QKV 投影；(2) 使用 KV 缓存的注意力计算，也称为自注意力；(3) 注意力输出的线性投影，称为输出投影；(4) 前馈网络 (FFN)。KV 缓存在推理过程中存储中间状态，即计算出的 K 和 V ，以避免为持续进行的请求重新计算历史状态。

Prefill and Decoding Phases. LLM 推理通过两个顺序阶段进行：预填充阶段和解码阶段。给定用户提示作为输入，预填充阶段处理整个提示以生成第一个输出令牌并填充 KV 缓存。然后，解码阶段通过使用最后一个输出令牌作为输入，迭代地生成后续输出令牌。累积的 KV 缓存允许解码阶段仅为新令牌计算 K 和 V ，从而提高效率。预填充阶段由于并行处理多个提示令牌而计算密集，其延迟由首令牌时间（TTFT）衡量。相比之下，解

²<https://github.com/ASISys/Adrenaline>

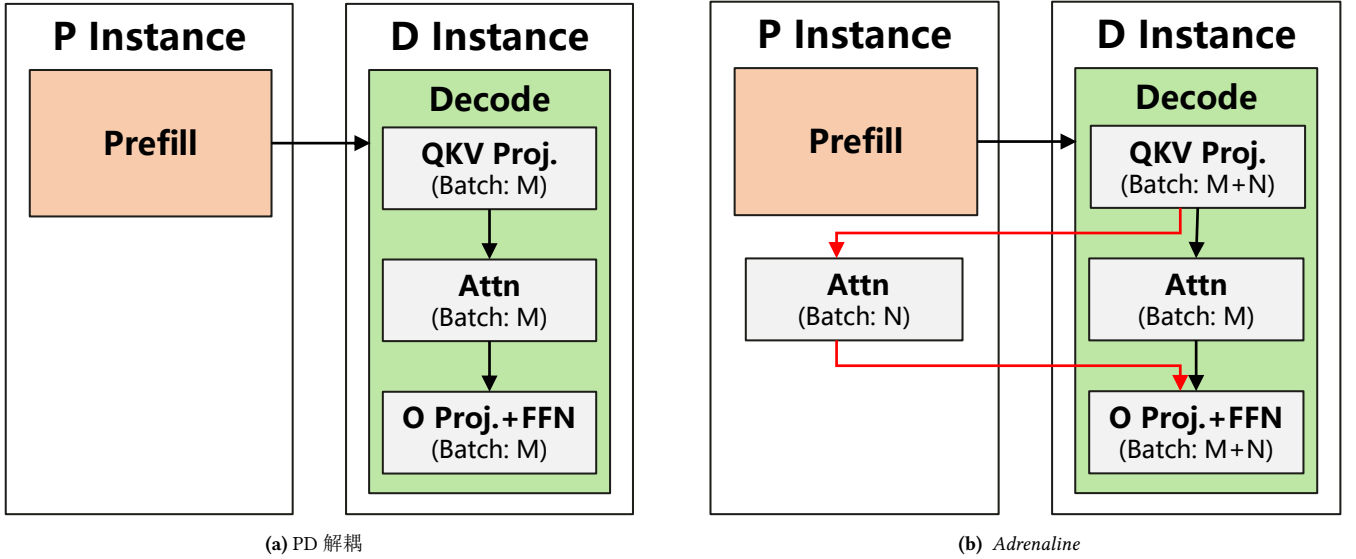


图 4. 原始 PD 解耦与 Adrenaline 的对比（解码批次大小：M 对比 M + N）。

码阶段是内存密集型的，这主要是由于频繁加载模型权重以及 KV 缓存大小的增长。其延迟由每个输出令牌的时间（TPOT）衡量。

2.2 Prefill and Decoding Disaggregation

将预填充和解码两个阶段放在相同的 GPU 上可以实现模型参数的共享，但这可能导致两个阶段之间的性能干扰。预填充和解码阶段的服务水平目标（SLO）存在显著差异。预填充的持久延迟（即 TTFT）通常长于解码阶段（即 TPOT）[32]。将计算密集型的预填充阶段与解码迭代同时执行会减慢解码阶段。此外，在实际的 LLM 推理场景中，准确预测序列长度和请求率是具有挑战性的。在系统初始化时静态分配 GPU 可能无法满足两个阶段的动态需求，从而导致 TTFT 和 TPOT 的 SLO 违规，或者由于过度配置而导致 GPU 资源利用不足。

鉴于预填充和解码阶段的显著不同特性，一些 PD 解耦设计 [20, 22, 32] 被提出来将这两个阶段分配到独立的 GPU 上。PD 解耦有效地消除了两个阶段的性能干扰，并能够根据需求灵活地扩展每个阶段的 GPU 资源。图 4 展示了现有 PD 解耦系统的基本架构。一个预填充或解码实例代表一组专门用于运行 LLM 推理的预填充或解码阶段的 GPU。一旦预填充阶段完成，相应的 KV 缓存就会通过高带宽互连（如 NVLink 和 InfiniBand）传输到解码实例。

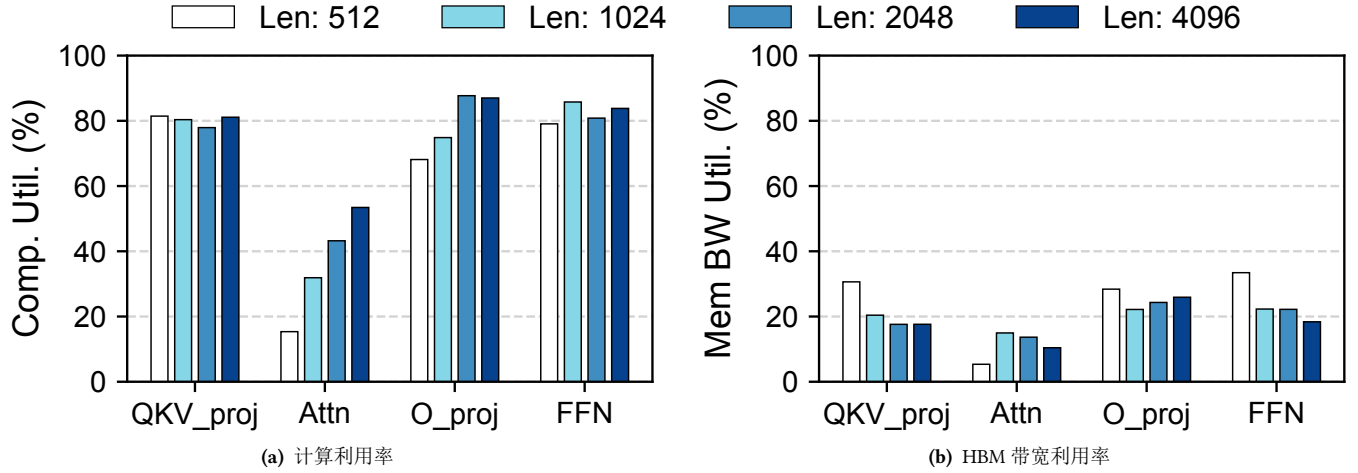


图 5. 在不同提示长度（批大小：1）下运行 Llama-2 7B 的预填充实例中主要内核的资源利用率。

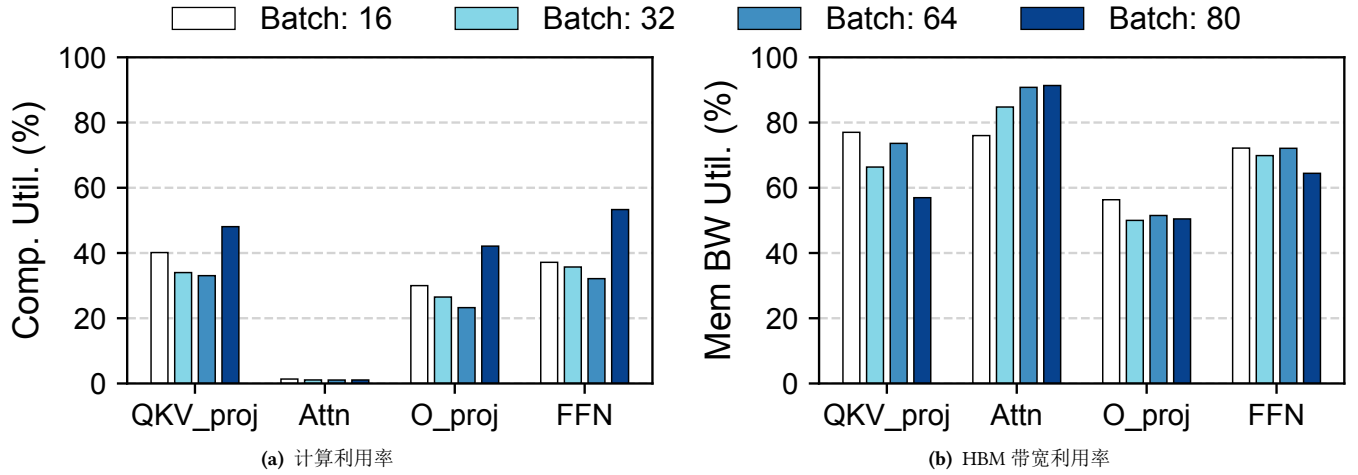


图 6. 在不同批大小（序列长度：1K）下运行 Llama-2 7B 的解码实例中主要内核的资源利用率。

2.3 Observation and Motivation

尽管 PD 解耦提供了灵活性，但我们观察到它会导致实例内资源利用率不足。如图 1 所示，预填充实例中的 HBM 带宽利用率和解码实例中的计算资源利用率都很低。为了进一步研究，我们测量了预填充和解码阶段底层内核的资源利用率。如图 5 和 6 所示，预填充阶段的四个关键内核——QKV 投影、注意力计算、输出投影和 FFN——与解码阶段相比是计算密集型的，导致内存带宽未得到充分利用。此外，请求的 KV 缓存一旦预填充阶段完成，会立即发送到解码实例，导致预填充阶段的 HBM 容量利用率较低（图 2）。与预填充阶段不同，解码阶段的自回归特性导致 GPU 内核的算术强度较低。因此，如图 6 所示，与预填充实例相比，四个解码内核的计算利用率要低得多。解码实例常常受到有限的 HBM 容量（图 2）和带宽（图 6b）的限制。在解码阶段，如图 3 所示，随着解码批大小的增加，注意力计算内核主导了执行时间。当批大小为 80 时，注意力内核占每个 Transformer 层执行时间的 69.5%。总的来说，注意力计算内核成为解码阶段 HBM 容量和带宽的主要消耗者。

为了提高预填充和解码实例中的 GPU 资源利用率，我们提出将解码阶段中部分内存密集型的注意力任务解耦并卸载到运行预填充阶段的 GPU 上执行。执行卸载的注意力任务可以提高预填充实例的容量和 HBM 带宽利用率。此外，这使得解码阶段的总批大小得以增加，从而提升解码实例的计算资源利用率。

然而，要在 *Adrenaline* 中实现高效的注意力卸载面临着三个主要挑战。

首先，注意力卸载在解码阶段的每个 Transformer 层内引入了额外的同步步骤。具体来说，如图 4b 所示，为了卸载注意力计算，解码实例必须将输入参数（即 *qkv*）发送给预填充实例。一旦卸载的注意力计算完成，其输出（即 *attn_out*）必须被发送回解码实例以进行后续处理。这些额外步骤——发送 *qkv*、执行卸载的注意力计算以及接收输出——必须与解码实例中的本地注意力计算仔细地重叠执行。其中每一步都涉及 CPU 干预，以向 GPU 发出相应的内核执行指令。每个 Transformer 层微小的同步开销累积起来会导致高昂的解码成本。例如，如果每层的同步开销为 0.5 毫秒，那么对于一个具有 32 层的 llama-2 7B 模型，其 TPOT 增加的总时间可能高达 16 毫秒。

其次，卸载的注意力任务会消耗预填充实例的 GPU 资源，可能导致预填充任务与卸载的注意力任务之间产生性能干扰。如图 1a 所示，预填充阶段的计算资源利用率已经很高。尽管注意力计算本身所需的计算能力相对较低，但注意力任务与预填充任务并发运行仍可能引起干扰。一个可能的解决方案是将一个 GPU 划分为两个虚拟 GPU (vGPU) [4]。然而，vGPU 是在虚拟机级别共享的，并且 vGPU 中预定义的资源分区配置对于现实场景中遇到的动态且不可预测的推理工作负载而言，通常效率低下。英伟达的多实例 GPU (MIG) [2] 技术为分割内存和计算资源（如流式多处理器 (SM)）提供了硬件支持。然而，MIG 划分的计算能力与分配的 HBM 容量成正比，这会极大地降低预填充性能。此外，它也面临着与 vGPU 类似的限制。总之，注意力任务与预填充任务的共址需要一个更灵活的 GPU 资源分区策略。

第三，准确确定要卸载的注意力数量（例如，图 4b 中的 N ）是困难的。卸载比例必须平衡预填充实例和解码实例中的计算与内存资源利用率。一种直接的方法是使用现有工作负载进行离线性能分析。然而，在线 LLM 推理的提示长度和输出长度是事先未知的，并且通常与先前的工作负载不一致。从离线分析中得出的静态卸载比例往往无法在具有动态工作负载的现实场景中表现良好。

3 The *Adrenaline* 设计

3.1 Overview

在本节中，我们介绍 *Adrenaline*，这是一种注意力解聚与卸载机制，旨在提升大语言模型服务系统的资源利用率和性能。*Adrenaline* 的核心思想是解聚解码阶段的部分注意力计算，并将其卸载到预填充实例。通过利用预填充实例的内存容量和带宽，*Adrenaline* 提高了它们的内存利用率，同时增加了解码批处理大小以提升解码实例的计算利用率。

Adrenaline 的概览如图 7 所示。与现有的 PD 解聚设计 [20, 22, 32] 类似，*Adrenaline* 包含三个主要模块：*proxy*、*prefill instance* 和 *decoding instance*。代理模块将推理任务路由到适当的预填充和解码实例，相应的引擎在大语言模型推理期间执行对应的预填充或解码任务。然而，与现有设计不同，*Adrenaline* 引入了解聚并将部分解码注意力计算卸载到远程的 *attention executor*，该执行器与预填充引擎共置。注意力执行器专门负责利用预填充实例中未充分利用的 GPU 内存资源来执行被卸载的解码注意力任务。

为了有效克服 §2.3 中概述的挑战，*Adrenaline* 在大语言模型推理工作流程中整合了三个关键优化。

首先，在解码阶段，*Adrenaline* 引入了一种低延迟解码同步机制 (§3.1)，以最小化远程注意力执行器与本地解码引擎之间的同步开销。

其次，通过利用资源高效的预填充共置设计 (§3.1.2)，*Adrenaline* 提高了预填充实例中的 GPU 内存利用率，同时确保预填充阶段有足够的计算资源。这消除了性能干扰，使系统能够满足所需的 SLO。

第三，*Adrenaline* 利用一种负载感知的卸载调度策略 (§3.2) 来自适应地决定一个请求的注意力计算是否应该被卸载。该策略考虑了可用 GPU 资源和当前负载等因素。

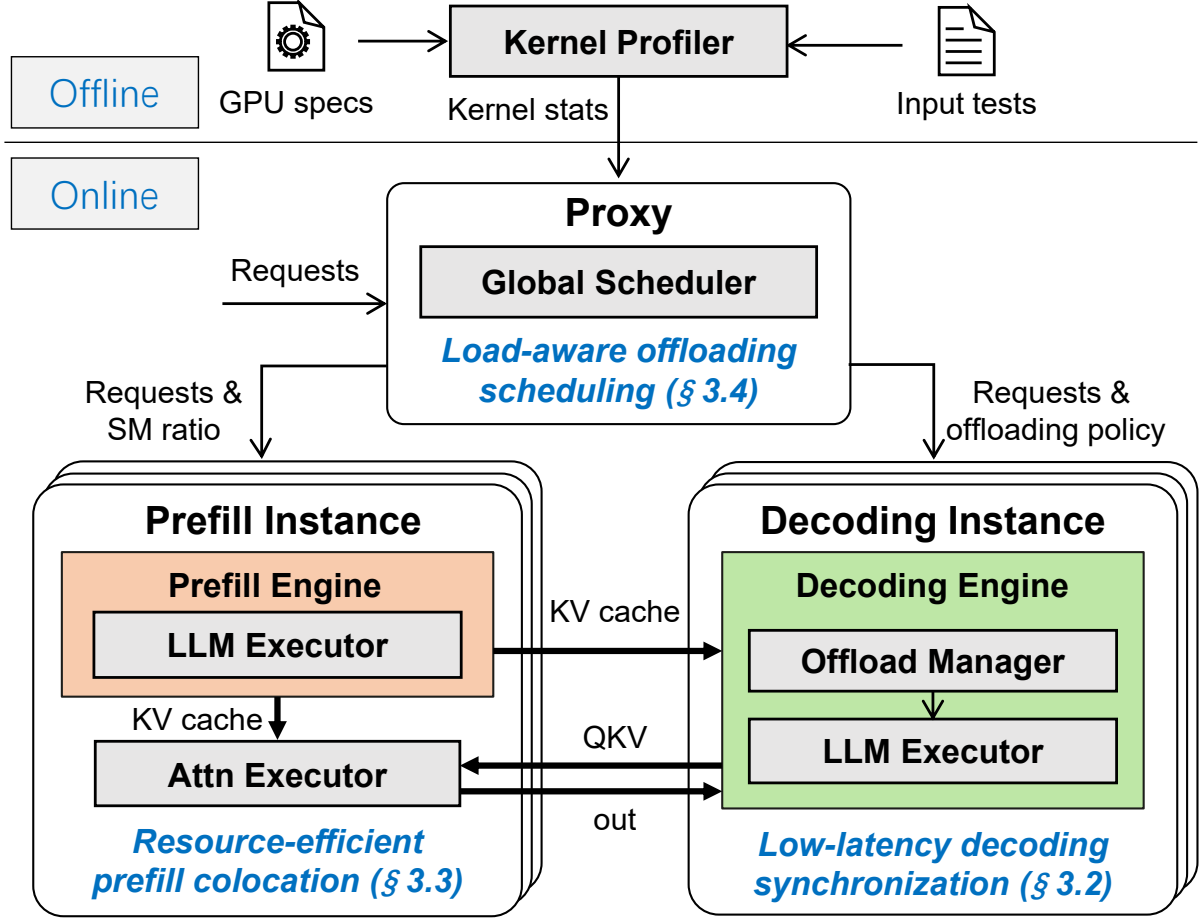


图 7. *Adrenaline* 的架构概览。

图 8 比较了使用和不使用注意力卸载时解码阶段的工作流程。在 *Adrenaline* 中 (8b)，预填充实例中的远程执行器与解码实例中的本地注意力计算并行执行被卸载的注意力计算，从而提高了并行吞吐量。然而，为了实现性能优势，关键在于最小化本地注意力执行与远程注意力卸载之间的操作调度和同步开销。如果管理不当，每层累积的开销会导致解码延迟增加，抵消吞吐量的潜在提升。为了解决这个问题，*Adrenaline* 通过精心优化注意力卸载工作流和内核启动，实现了低延迟解码同步。

3.1.1 Offloading Workflow Orchestration. 注意力卸载工作流的关键路径包括从发送 qkv 开始到接收注意力输出为止的操作。如果关键路径比在解码实例中执行本地注意力更长，推理就会因为等待卸载注意力的输出而停滞。

为了缩短关键路径，*Adrenaline* 精心组织了注意力卸载工作流。首先，卸载注意力的元数据和内存管理操作被移出关键路径 (图 8b 中的 ❶)。当代理确定某个请求的注意力需要被卸载时 (算法 1)，新请求会作为提示发送给注意力执行器。这使得注意力执行器能够在进入关键路径之前为新请求初始化运行时元数据 (例如，提示长度)。类似地，资源管理操作，如分配和回收缓存块，也在关键路径之外执行。其次，*Adrenaline* 不是分别发送分散的注意力输入 (例如 q 、 k 、 v)，而是将它们分组并在单次操作中发送聚合数据 (图 8b 中的 ❷)。这减少了注意力执行器输入的通信成本。第三，代理中的调度算法 (§3.2) 确保注意力执行器中注意力内核花费的 GPU

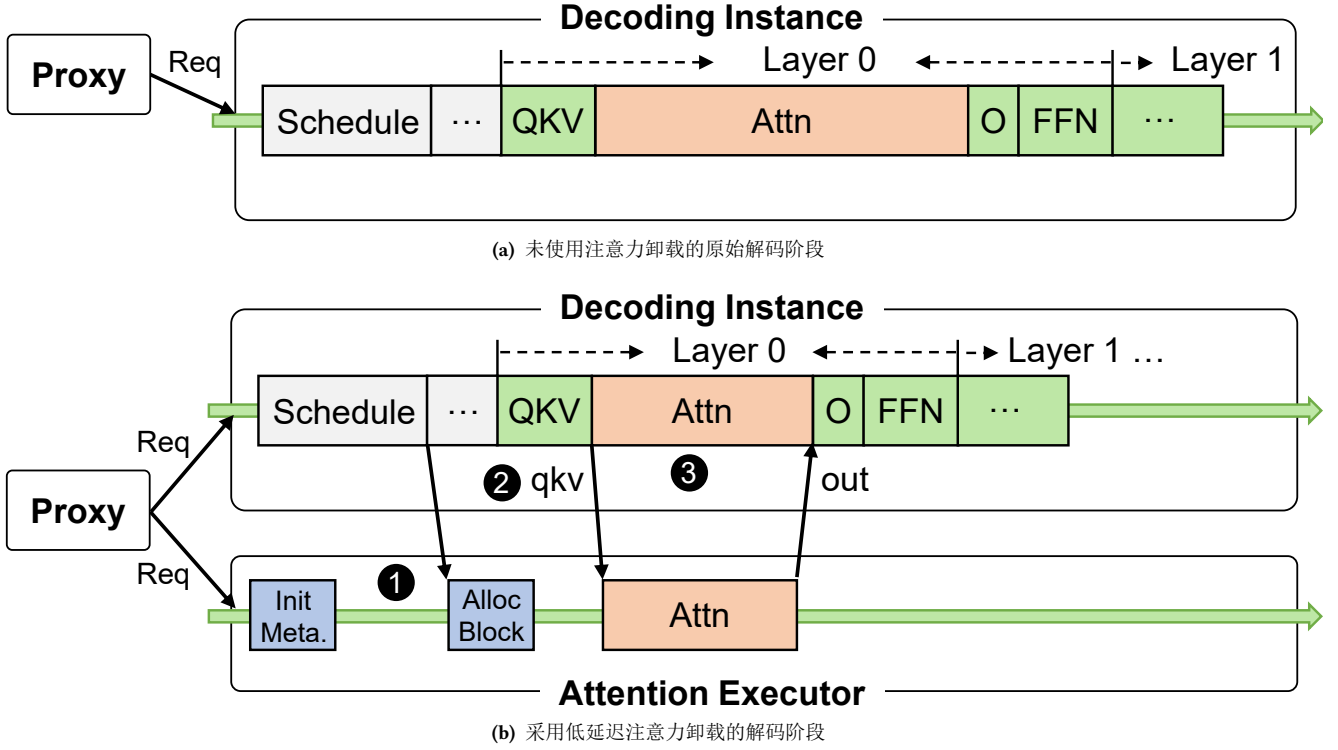


图 8. 在 *Adrenaline* 中，对相同批次大小的请求进行解码时，不使用/使用注意力卸载的对比。

时间与解码实例中本地注意力内核花费的时间重叠（图 8b 中的 ③）。这减少了空闲等待时间，提高了 GPU 资源利用率。

3.1.2 Kernel Pre-launching. 与现有的 PD 解耦系统类似，注意力卸载涉及每个 Transformer 层的多个小型 GPU 内核，例如分组 qkv 、发送 qkv 、接收注意力输出以及合并来自两个注意力内核的输出。尽管这些小型内核在 GPU 上执行迅速，但 CPU 为顺序启动它们所产生的开销可能不可忽视。例如，当使用一个 A100 执行 Llama-2 7B 的解码阶段时（批大小为 8，序列长度为 1K），每个 Transformer 层的平均 GPU 时间为 0.38 毫秒，但平均 CPU 时间为 1.137 毫秒，因此每个 Transformer 层因 CPU 开销浪费了 0.76 毫秒的 GPU 时间。与其逐个启动小型内核，CUDA 图技术 [1] 捕获多个顺序内核，将其作为单个融合内核重放，从而有效提升解码性能（例如，在上述设置下提升 2.6 $\times$ ）。然而，vLLM 中现有的 CUDA 图技术对于注意力卸载变得无效，因为本地执行请求和卸载请求的未知且动态的数量违反了 vLLM CUDA 图对静态张量形状的要求，这极大地复杂化了图捕获操作。

为了缓解内核启动开销，*Adrenaline* 为注意力卸载采用了二维 CUDA 图。第一维指的是解码实例中原始批大小的图容量 (C_d)。第二维表示卸载注意力计算任务的图容量 (C_o)。这两个参数的组合导致了 $C_d \times C_o$ 个 CUDA 图，增加了存储开销。为了解决这个问题，*Adrenaline* 设置了可配置的间隔来限制捕获的 CUDA 图的总数。

给定一个卸载比率 (§3.2)，一个关键问题是为每个解码迭代选择一个已捕获的 CUDA 图。通过利用调度器的输出，解码实例中的卸载管理器采用能同时容纳本地和远程注意力批次的最小二维 CUDA 图。选定的二维 CUDA 图由配对的解码实例和注意力执行器使用。为了降低同步成本，选定 CUDA 图的结果在模型前向执行的关键路径之外发送给注意力执行器（图 8b 中的 ④）。

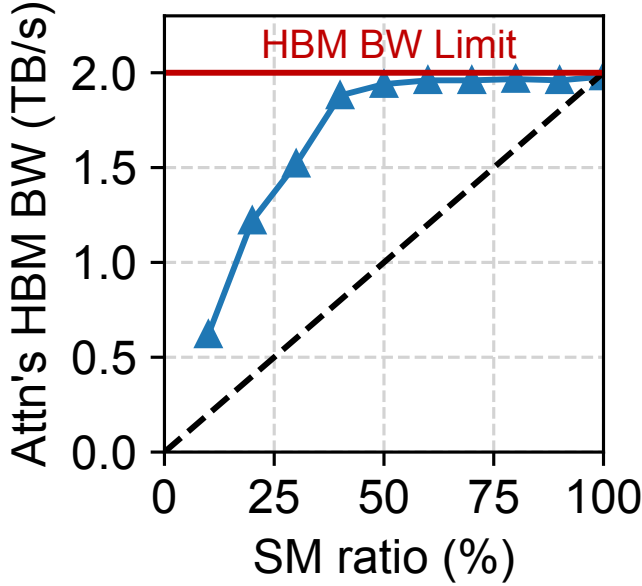


图 9. 使用不同比例 SM 时注意力内核的最大内存带宽.

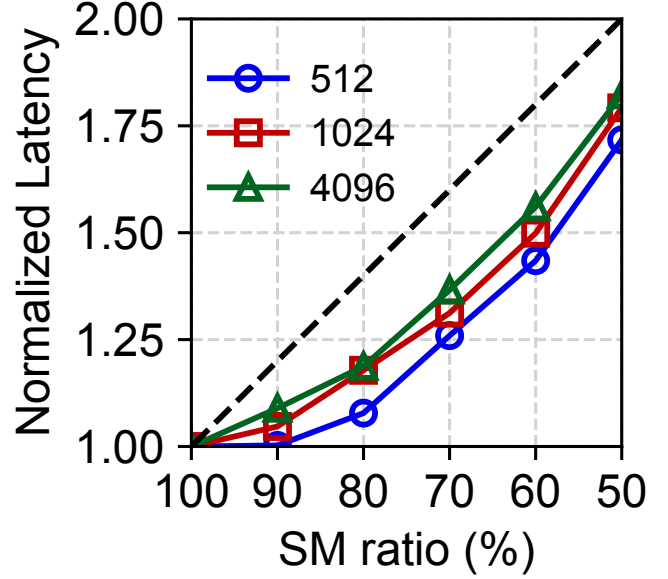


图 10. 使用不同比例 SM 时, 不同提示长度下的归一化预填充吞吐量.

卸载注意力由注意力执行器执行, 该执行器与预填充实例共享 GPU 资源。虽然这种共置提高了 GPU 利用率, 但也带来了挑战, 例如潜在资源竞争, 这可能导致性能干扰。

为了解决潜在的性能干扰问题, 我们对资源受限的 LLM 推理性能进行了研究。基于上述研究的关键观察, 我们在预填充实例中提出了一种资源高效的共置机制, 以确保性能隔离并提高整体系统性能。

3.1.3 Resource Utilization Profiling. 为了研究资源划分对 LLM 推理的影响, 我们评估了在不同计算资源 (SM) 比例下的预填充阶段和注意力计算。具体而言, 我们采用 NVIDIA MPS 技术 [3] 来为目标计算任务限制 SM 的数量。在本研究中, 我们获得了以下两个观察结果。

1) *With the increase of used SMs, the HBM bandwidth utilization of the attention computation kernel super-linearly increases.* 我们使用 Llama-2 7B 模型测量了不同 SM 比例下注意力计算的 HBM 带宽。如图 9 所示, 由于注意力计算内核是内存密集型的, 即使部分 SM 也能够实现较高的内存带宽。由于现代 GPU 中的多级并行性 (例如, SM 并行、warp 并行和线程并行执行), 注意力内核表现出卓越的效率。例如, 20% 个 SM 获得了 A100 HBM 带宽的 60%。

2) *With the decrease of used SMs, the prefill latency increases at a sub-linear level.* 我们限制了 SM 核心, 并使用 Llama-2 7B 模型评估了预填充延迟。图 10 表明, 预填充性能与分配的 SM 数量密切相关。在 PD 解耦系统中, 预填充阶段中的一些子步骤 (例如, 请求路由、调度和 KV 缓存传输) 不依赖于 GPU 计算资源。因此, 在部分 SM 下限制预填充阶段的性能下降是可能的。

总之, 关键的洞察是 SM 数量与协同定位组件 (即注意力带宽和预填充延迟) 性能之间的非线性关系, 这确保了预填充实例中资源共享的效率。

3.1.4 Adaptive Resource Partition. 基于上述两个观察, *Adrenaline* 采用了一种资源高效的方案来协同放置预填充引擎和注意力执行器。其核心思想是根据 TTFT 服务等级目标 (SLO) 对流式多处理器 (SM) 进行分区。性能隔离分为两个阶段: *offline profiling* 和 *online serving*, 具体细节如下。

在离线分析阶段，内核分析器会测量不同提示词长度在不同 SM 数量下的预填充延迟。由于预填充阶段是计算密集型的，特定模型的执行时间与所涉及的浮点运算（FLOP）成正比，而 FLOP 又与所有提示词的总长度相对应。

在线服务阶段，*Adrenaline* 提供了一个可配置变量来设置分配的 SM 比例。根据 TTFT SLO 以及离线分析阶段获得的统计数据，*Adrenaline* 计算出最小的 SM 比例，该比例随后在 NVIDIA MPS 中用于限制后续预填充阶段的计算资源。

3.2 Load-aware Offloading Scheduling

对于，关键的设计目标是实现最优的性能提升，同时最小化卸载开销。在卸载策略中需要解决两个主要问题：

(1) *How many decoding attention tasks can be offloaded?* 过低的卸载比例会使得预填充实例中的空闲 GPU 内存资源未得到充分利用，导致整体性能提升微乎其微。相反，卸载过多的解码注意力任务会因过度的同步开销，在解码阶段导致生成停滞，从而阻碍整体性能。

(2) *Given an ideal offloading ratio, how to efficiently determine whether offloading is necessary for a request in a dynamic workload?* 在实际场景中，不同用户请求的提示词和生成输出长度各不相同。此外，用户请求的到达率也可能波动。因此，设计一个低开销、细粒度的调度机制至关重要，该机制需要能够适应这些动态条件并做出高效的卸载决策。

针对第一个问题，我们建立了一个模型来获取卸载比例的上限 (§3.2.1)。针对第二个问题，*Adrenaline* 利用代理中的运行时信息来实现均衡的注意力卸载 (§3.2.2) 和细粒度的自适应调度 (§3.2.2)。

3.2.1 Bounding Offloading Ratio. 为了解决确定合适卸载比例的第一个问题，我们的目标是找到一个界限，该界限能不影响系统效率的前提下最大化注意力卸载的收益。核心思想是找出解码批次大小可以增加多少而不对 TPOT 产生负面影响，因为低延迟对于 PD 解耦系统的 SLO 达成至关重要。具体来说，如图 4b 所示，给定解码实例中有 M 个解码请求，问题在于找到不影响 TPOT 的额外请求 N 的最大值。*Adrenaline* 考虑了预填充实例中用于内存密集型注意力内核的内存资源，以及解码实例中用于非注意力内核的可用计算资源。

就注意力内核而言，卸载比例的上限受限于预填充实例中可用的 HBM 容量和带宽，以实现远程注意力计算的完美重叠。设分配给每个解码实例的平均预填充实例数量为 n 。 HBM_{pi} 和 BW_{pi} 表示分配给注意力执行器的预填充实例 i 的 HBM 容量和带宽。 HBM_d 和 BW_d 表示解码实例中相应的内存资源。在实践中， HBM_{pi} 、 HBM_d 、 BW_{pi} 和 BW_d 由离线阶段的内核分析器获得。就内存资源而言的卸载比例上限，记为 $OB_{mem}(n)$ ，可计算如下：

$$OB_{mem}(n) = \min\left(\frac{\sum_{i=1}^n HBM_{pi}}{HBM_d}, \frac{\sum_{i=1}^n BW_{pi}}{BW_d}\right) \quad (1)$$

对于非注意力内核，增加解码批次大小也会消耗解码实例中更多的计算资源，这导致了卸载比例的第二个上限。在解码阶段，非注意力内核（如用于 QKV 和 O 的 FFN 和线性投影）的算术强度（即 FLOP:I/O 比率）由下式给出： $O(1/(1/h + 1/b))$ ，其中 h 表示隐藏状态大小（一个模型特定的常数）， b 是批次大小。随着解码批次大小的增加，算术强度也随之增加。然而，当非注意力内核保持内存受限时，这些非注意力内核的执行时间在不同批次大小下是稳定的。非注意力内核在内存受限和计算受限之间的算术强度边界由 GPU 硬件特性（例如，FLOPs 和内存带宽）和模型参数（例如， h ）决定。为了准确识别这一边界，需要进行离线分析以确定非注意力内核的执行时间随批次大小增加而增加的批次大小。设 B_{max} 表示内存受限非注意力内核的最大批次大小。给定一个 TPOT SLO， B_{TPOT} 表示解码实例在不进行卸载的情况下可以处理的最大批次大小，这是在运行时由代理收集的。基于

Algorithm 1 Load-aware Offloading Scheduling Algorithm

Require: a new request req , the upper bound of offloading ratio $OB(n, B_{max})$, local running requests in the decode instance LR , offloaded requests of the decode instance OR

Ensure: flag $need_offload$

```
1:  $need\_offload \leftarrow 0$ 
2:  $attn\_max\_tokens \leftarrow \sum_{r \in OR} r.max\_token$ 
3:  $attn\_used\_tokens \leftarrow \sum_{r \in OR} r.used\_token$ 
4:  $decode\_used\_tokens \leftarrow \sum_{r \in LR} r.used\_token$ 
5: if  $attn\_used\_tokens + req.max\_token <$   
    $decode\_used\_tokens \times OB(n, B_{max})$  then ▷ C1
6:    $need\_offload \leftarrow 1$ 
7: else if  $(attn\_used\_tokens + req.used\_token <$   
    $decode\_used\_tokens \times OB(n, B_{max}))$   
    $\&\&(|OR| + 1 < |LR| \times OB(n, B_{max}))$  then ▷ C2
8:    $need\_offload \leftarrow 1$ 
9: end if
10: Return  $need\_offload$ 
```

这些测量结果，就计算资源而言的卸载比例上限可计算如下：

$$OB_{comp}(B_{max}) = \frac{B_{max} - B_{TPOT}}{B_{TPOT}} \quad (2)$$

总体而言，考虑到内存和计算资源的双重约束，卸载比例的上限由以下方程给出：

$$OB(n, B_{max}) = \min(OB_{mem}(n), OB_{comp}(B_{max})) \quad (3)$$

$OB(n, B_{max})$ 是总体上限，由 $OB_{mem}(n)$ 和 $OB_{comp}(B_{max})$ 中的最小值决定。 $OB_{mem}(n)$ 的值来源于离线性能分析，并在预填充实例分配的资源发生变化时更新。另一方面， $OB_{comp}(B_{max})$ 是一个动态阈值，取决于诸如工作负载特性（例如序列长度和请求速率）以及目标 TPOT SLO 等因素。

3.2.2 Load Awareness. 实现卸载策略的一种直接方法是在每个解码实例的调度器内部进行管理。然而，解码实例中的本地调度器无法感知预填充实例的动态扩缩容，后者可以根据需求进行扩展或收缩。

Adrenaline 通过在其代理中采用一个 *global scheduler* 来解决此问题，该代理管理运行时信息的元数据，称为 *runtime metadata*，以高效支持动态工作负载下的卸载。通过路由请求和响应，代理可以轻松跟踪解码阶段中活跃请求的数量及其对应的序列长度。借助运行时元数据，代理能够洞察解码实例的当前负载，从而计算 B_{TOPT} ，并进而计算卸载比率上限 $OB_{comp}(B_{max})$ 。代理还能感知可用的预填充实例，这使其能够在预填充实例添加或移除时更新 $OB_{mem}(n)$ 。

为了确定一个请求是否应该被卸载，*Adrenaline* 在自适应调度策略中遵循一个原则：仅当卸载比率在相应的上限 $OB(n, B_{max})$ 之内时，才卸载该请求。将过多任务卸载到远程注意力执行器会使预填充实例过载，并增加解码阶段的同步开销。

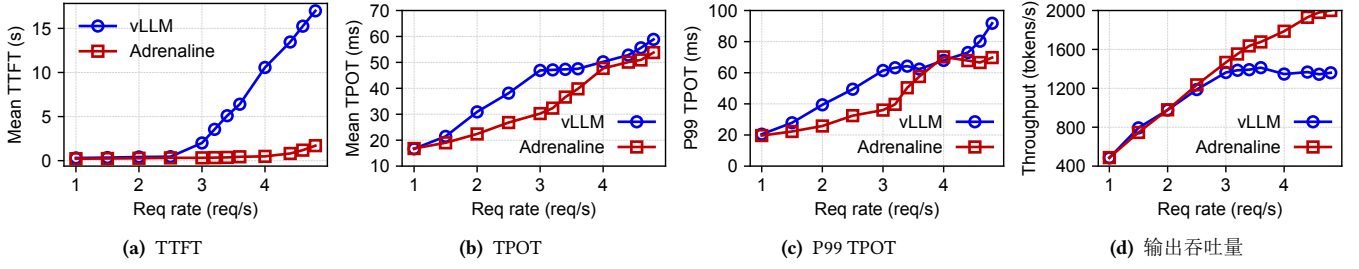


图 11. 使用 Llama-2 7B 模型的 ShareGPT 端到端性能 (*Adrenaline* 的卸载比例: 70%)。

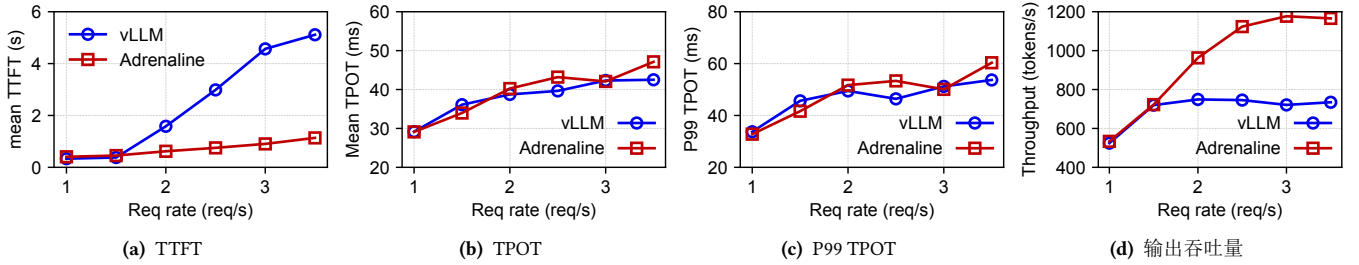


图 12. 使用 Llama-2 13B 模型的 ShareGPT 端到端性能 (*Adrenaline* 的卸载比例: 70%)。

Adrenaline 利用代理中的运行时元数据来自适应地调度每个请求。调度算法的概述见算法 1。遵循所提出的原则，如果不会增加解码延迟，则通过设置 *need_offload* = 1 来启用注意力卸载。具体来说，对于新请求，允许卸载存在两种替代条件：

- **C1:** 运行现有卸载请求和新请求的注意力计算，即使使用最大序列长度，也不会超过卸载比例限制。对序列长度的这种检查确保了注意力执行器中计算时间的最大值可以被本地注意力运行时间所重叠，从而为注意力卸载实现低延迟同步。
- **C2:** 对于注意力执行器和解码实例，当前序列长度的比例和注意力批次大小均在卸载比例限制内。由于解码实例和相应的注意力执行器以相同频率执行，检查注意力批次大小可确保注意力执行器的负载不会在后续迭代中引入解码同步成本。

如果满足条件 C1 或 C2，则新请求的注意力可以卸载，而不会增加解码延迟。

4 Evaluation

4.1 Experimental Setup

Testbed. 我们的实验在一台高性能服务器上进行，以评估 *Adrenaline* 在 LLM 推理中的效率。该服务器包含八块通过 600 GB/s NVLink 连接的 NVIDIA A100-80GB SMX GPU。

Models. 我们的评估使用了流行的 Llama-2 [25] 7B 和 13B 模型，采用默认精度 (float16)。

Baseline. *Adrenaline* 基于 vLLM [16] 实现，vLLM 是一个广泛使用的开源 LLM 推理框架。默认情况下，我们使用 vLLM 作为性能比较的基线。所评估的 vLLM 版本 (v6.3.0) 已经支持 PD 解耦。

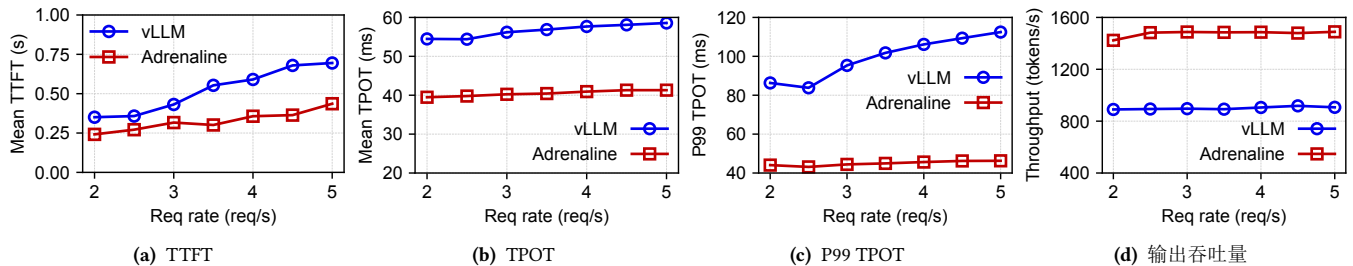


图 13. 使用 Llama-2 7B 的 OpenThoughts 端到端性能 (*Adrenaline* 的卸载比例: 80%)。

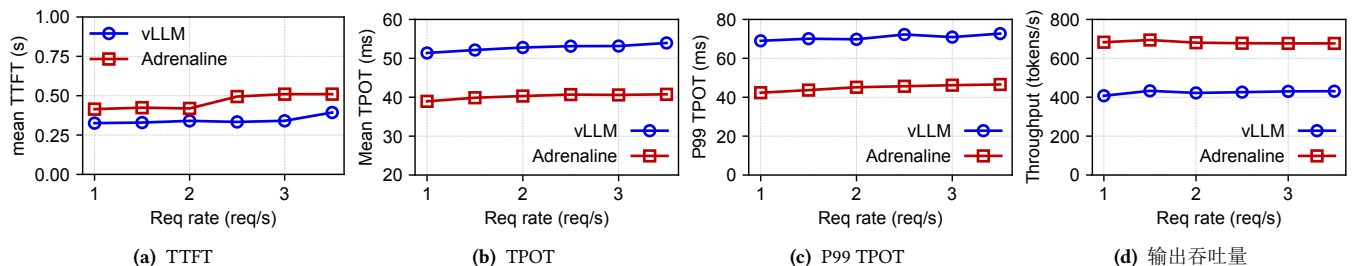


图 14. 使用 Llama-2 13B 的 OpenThoughts 端到端性能 (*Adrenaline* 的卸载比例: 80%)。

Workloads. 我们使用真实世界的工作负载来评估性能，包括 ShareGPT [5] 和 OpenThoughts [7]。ShareGPT 代表聊天机器人场景，已被相关的 LLM 推理研究所采用 [12, 32]。OpenThoughts [7] 是近期新兴的推理模型的一个流行数据集 [10, 31]。两种工作负载都包含可变长度的提示，反映了真实世界 LLM 推理工作负载的动态特性。通常，由于生成输出开头的思维链 [28]，OpenThoughts 中的输出/提示比率大于 ShareGPT。

Metrics. 在评估中，TTFT 和 TPOT 分别用于衡量预填充和解码延迟。请注意，vLLM 的 PD 解耦版本中的 TTFT 包括将 KV 缓存从预填充实例传输到解码实例的时间。*Adrenaline* 遵循 vLLM，采用了相同的 TTFT 指标定义。此外，我们评估了解码阶段的令牌生成吞吐量，称为 *output token throughput*。为了避免系统预热阶段的影响，我们在比较的系统达到稳定平衡状态时记录并报告输出令牌吞吐量，这反映了不同系统的最佳生成吞吐量。具体来说，我们将解码实例的 HBM 容量达到饱和（例如，vLLM 中的抢占）的第一次和最后一次之间的持续时间视为稳定状态。如果请求速率太低，无法使解码实例的 HBM 空间饱和，我们将在解码批次大小达到其峰值批次大小的 80% 时报告平均输出令牌吞吐量。

4.2 End-to-End Performance

在本节中，我们评估了不同方案的端到端性能，以展示 *Adrenaline* 在真实工作负载下的效率。

我们测量了 7B 和 13B 模型在不同请求率下的 TTFT，如图 11-14 所示。对于图 11a 和 13a 中的 ShareGPT，当请求率较低时，*Adrenaline* 的 TTFT 与 vLLM 接近。然而，通过提高 ShareGPT 的请求率，vLLM 的解码实例会耗尽 HBM 资源，从而阻塞新的解码请求并显著延长排队时间。例如，当使用 Llama-2 7B 模型且 ShareGPT 的请求率为 4 时，vLLM 的 TTFT 比 *Adrenaline* 高出 22 $\times$ 。通过卸载一些内存密集型的注意力计算任务，*Adrenaline* 增加了解码阶段的最大批次大小。因此，*Adrenaline* 中的排队时间得到缓解，TTFT 得以降低。由于 OpenThoughts 中请求的提示长度较短，两种方案的 TTFT 相似。

TPOT. 如图 11b-14b 所示，在相同请求率下，*Adrenaline* 的 TPOT 通常接近或低于 vLLM。在 TPOT 方面，*Adrenaline* 的收益来自两个因素。首先，当请求率较低时（例如，图 11b 中请求率 < 3），*Adrenaline* 利用预填充实

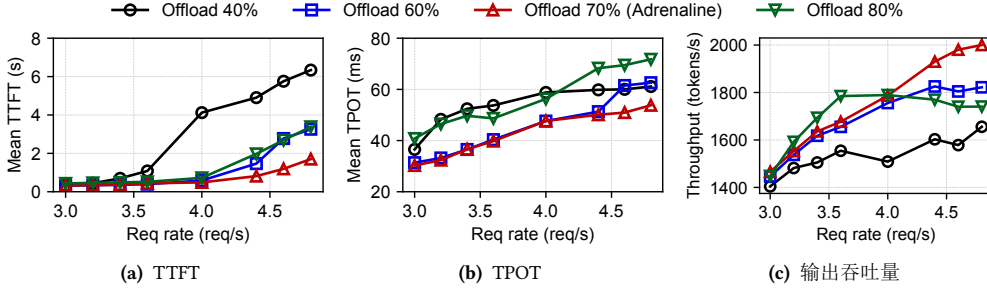


图 15. 使用不同卸载比例的 Llama-2 7B 在 ShareGPT 上的性能。

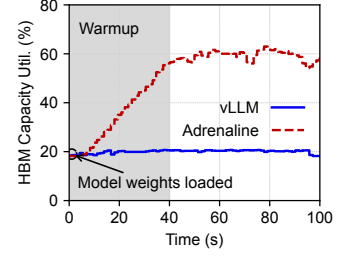


图 16. 预填充实例中的 HBM 使用情况。

例的闲置 HBM 带宽来执行卸载的注意力计算任务。聚合的 HBM 带宽减少了注意力的执行时间，而注意力是大批次解码阶段的瓶颈（图 3）。然而，如果解码实例的 HBM 空间耗尽，*Adrenaline* 中卸载注意力的批次大小将受限于本地注意力计算任务的执行时间（§3.2）。因此，当请求率高于图 11b 中的 3 或图 13b 中的 1.5 时，对 TPOT 的收益会减弱。其次，*Adrenaline* 减轻了抢占（或交换）的影响，当解码实例的 HBM 容量不足以容纳正在进行的请求所生成的输出时，就会发生抢占。被抢占的请求必须至少等待一个先前请求完成以释放 HBM 空间后才能被调度。对于输出序列较长的工作负载（例如 OpenThoughts），抢占开销很高。如图 13b 和 14b 所示，与 vLLM 相比，*Adrenaline* 减轻了抢占开销，从而将 OpenThoughts 的 7B 和 13B 模型的平均 TPOT 分别降低了 26.9%-29.5% 和 23.4%-24.5%。

第 99 百分位数 (P99) 的 TPOT 也使用真实世界工作负载进行评估。与平均 TPOT 类似，低延迟解码同步确保 P99 TPOT 接近 vLLM 的水平，如图 11c 和 13c 所示。对于 OpenThoughts，由于聚合的 HBM 带宽和解码抢占的缓解，与 vLLM 相比，7B 和 13B 模型的 P99 TPOT 分别降低了 48.5%-58.8% 和 34.9%-38.7%。

Output Token Throughput. 我们使用不同的请求率来评估输出令牌的吞吐量。如图 11d-14d 所示，两种方案在低请求率下实现了相似的吞吐量。vLLM 的吞吐量由于解码实例中有限的 HBM 空间和带宽而达到平台期。*Adrenaline* 通过利用预填充实例的 HBM 资源来提高吞吐量，从而在 ShareGPT 数据集上为 Llama-2 7B 实现了高达 1.47× 的加速（Llama-2 13B 为 1.63×）。对于 OpenThoughts 数据集，*Adrenaline* 卸载了更多的注意力（80%），并为 Llama-2 7B 获得了 1.60×-1.66× 的加速（Llama-2 13B 为 1.57×-1.68×）。

图 15 展示了 ShareGPT 使用不同卸载比例时的端到端性能。当卸载比例较低时，*Adrenaline* 随着卸载比例的增加提升了推理性能。如图 15 所示，卸载 80% 注意力的性能低于卸载 70% 的比例，这是由于注意力执行器的运行时间较长，无法被解码实例的本地注意力计算所重叠。*Adrenaline* 根据离线性能分析获得的内核统计信息以及从运行时元数据收集到的工作负载模式，为注意力卸载确定了拐点，从而自动为不同的工作负载实现高性能。

4.3 Resource Utilization

在本节中，为了展示 *Adrenaline* 在解决 GPU 资源利用率不均衡问题上的效率，我们评估了预填充实例中的 HBM 容量和带宽利用率以及解码实例中的计算资源利用率。HBM 容量利用率的评估使用 Llama-2 7B 模型，其他评估则使用不同模型进行。所有实验均使用 ShareGPT 工作负载。

HBM Capacity Utilization. 如图 16 所示，加载模型权重后，vLLM 中预填充实例的 HBM 容量利用率约为 20%，剩余的 HBM 容量未被使用。*Adrenaline* 的注意力执行器利用预填充实例中未使用的 HBM 容量来存储卸载注意力的 KV 缓存。因此，图 16 显示，与默认的 PD 解耦方案相比，*Adrenaline* 在预热阶段后，预填充实例中的 HBM 利用率达到了 2.28×。

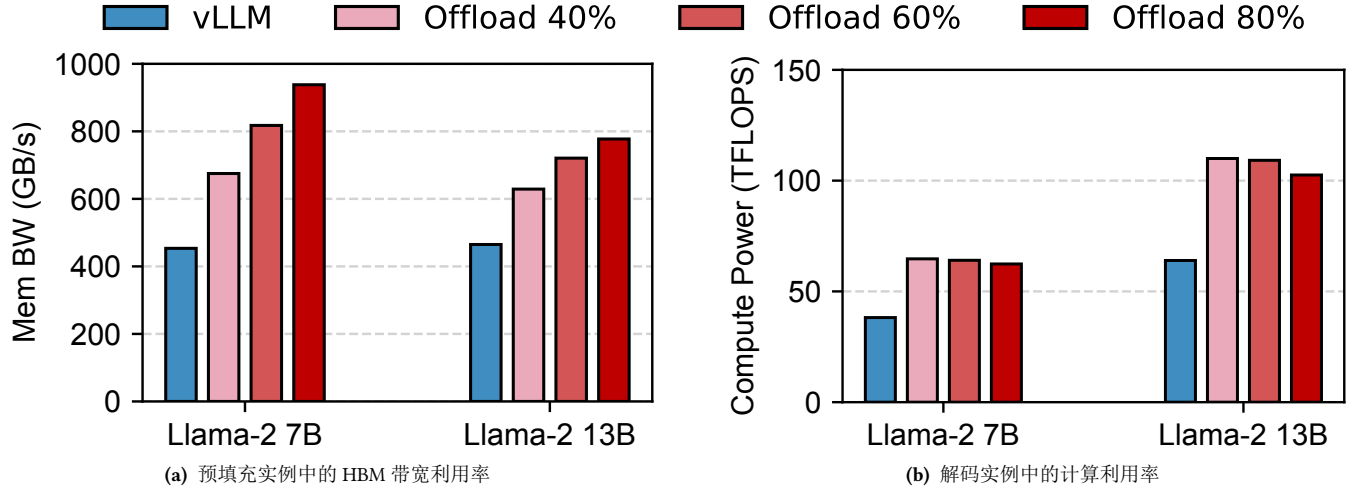


图 17. 不同卸载比例下的资源利用率。

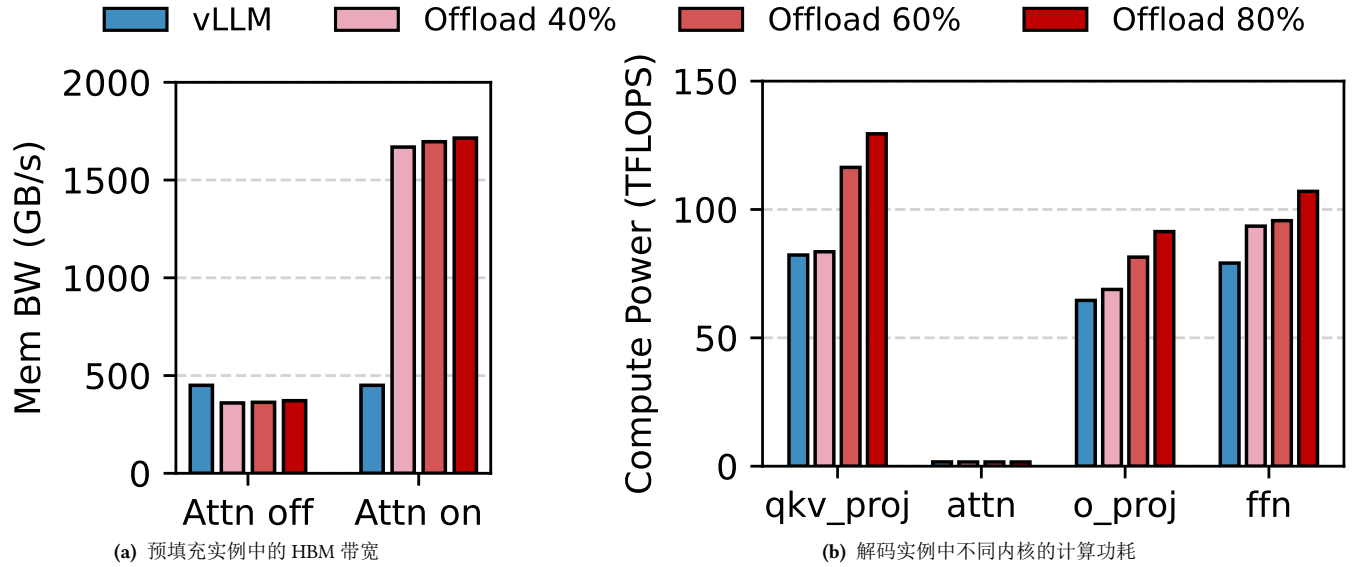


图 18. 资源利用率分解。(Attn on 表示运行卸载注意力时的阶段)

HBM Bandwidth Utilization. 图 17a 表明，卸载注意力机制改善了预填充实例的 HBM 带宽，因为注意力执行器计算的是聚合的内存密集型注意力任务。具体来说，与基线相比，*Adrenaline* 在预填充实例中为 Llama-2 7B 实现了 1.49x-2.07x 的 HBM 带宽利用率。Llama-2 13B 的趋势类似（与 vLLM 相比为 1.37x-1.93x）。为了研究 HBM 带宽消耗，我们分析了注意力执行器运行（记为 Attn on）和空闲（记为 Attn off）时预填充实例的内存带宽。如图 18a 所示，在默认的 PD 解耦方案中，注意力执行器平均比预填充实例多获得 3.76x 的 HBM 带宽（带宽容量限制的 83.0%）。需要注意的是，ShareGPT 仅涉及注意力执行器运行时间的 23.0%-42.9%。卸载更多 KV 缓存的工作负载会增加注意力执行器的运行时间比例，从而在预填充实例中消耗更多的 HBM 带宽。

Compute Power Utilization. 如图 17b 所示，通过解耦注意力计算，*Adrenaline* 增加了解码批次大小，因此与 vLLM 相比，为 Llama-2 7B 实现了 1.67x 的计算能力（Llama-2 13B 为 1.68x）。请注意，当卸载比例在 40% 到 80% 之间时，分配的 CUDA 核心仍然能够容纳增加的批次大小，这导致图 17b 中 *Adrenaline* 的计算能力保持不

变。在解码阶段内，主要 GPU 内核的计算负载与批次大小成正比。图 18b 展示了四个主要内核的计算能力分解。随着卸载比例的增加，三个非注意力 GPU 内核（即 QKV 投影、O 投影和 FFN）的计算负载也会增加。具体来说，当卸载 40% 的注意力时，*Adrenaline* 在三个非注意力内核中的平均计算能力提高了 8.8%。如果卸载比例为 80%，与 vLLM 相比，平均改进变为 44.7%。

5 Related Works

Disaggregated LLM Serving. 最近的研究工作提倡采用预填充和解码分离来实现高效的大语言模型服务。Dist-Serve [32] 将预填充和解码计算分配给独立的 GPU 以减少干扰，并为每个阶段独立配置资源。Splitwise [21] 通过综合考虑成本、吞吐量和功耗来优化分离式大语言模型服务系统。Mooncake [23] 利用 CPU、DRAM 和 SSD 资源，在分离式大语言模型服务系统中实现了一个分离的 KV 缓存池。本文揭示了分离式大语言模型服务系统中的低资源利用率问题，并提出 *Adrenaline* 通过注意力分离和卸载来解决该问题。

Attention Offloading. 最近的一些工作探索了注意力卸载在提升异构大语言模型服务系统性能方面的潜力。例如，Lamina [9] 将注意力操作从高端加速器卸载到一组为注意力计算优化的低成本内存设备（如消费级 GPU）池中。一些设计 [13, 15] 提出将解码注意力计算卸载到 CPU。NEO [15] 将部分注意力计算和 KV 缓存状态从 GPU 卸载到 CPU 机器，从而增加 GPU 批处理大小并提高推理吞吐量。InstInfer [19] 利用计算存储设备来设计专用的存储内注意力计算引擎，减少了传输 KV 缓存的开销。与这些依赖额外硬件资源进行注意力卸载的方法不同，我们提出的 *Adrenaline* 通过利用大语言模型服务系统内未充分利用的资源来提升推理性能，实现了无需额外硬件的高效注意力卸载。

6 Conclusion

本文针对采用预填充-解码（PD）解耦架构的 LLM 服务系统中的 GPU 资源利用率低下问题，提出了 *Adrenaline*，一种注意力计算解耦与卸载机制，通过将解码注意力任务卸载到预填充实例来提升内存和计算利用率。关键技术包括低延迟同步、资源高效的预填充共置以及负载感知的卸载调度。实验结果表明，*Adrenaline* 显著提高了 GPU 利用率，增加了吞吐量，并满足了服务水平目标，为优化 LLM 推理性能提供了一个可扩展且经济高效的解决方案。

References

- [1] 2024. NVIDIA CUDA Graph. <https://developer.nvidia.com/blog/cuda-graphs/>.
- [2] 2024. NVIDIA Multi-Instance GPU. <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>.
- [3] 2024. NVIDIA Multi-Process Service Manual. <https://docs.nvidia.com/deploy/mps/index.html>.
- [4] 2024. NVIDIA virtual GPU solutions. <https://www.nvidia.com/en-us/data-center/virtual-solutions/>.
- [5] 2024. ShareGPT. <https://sharegpt.com/>.
- [6] 2025. NVIDIA virtual GPU solutions. <https://www.nvidia.com/en-us/ai/dynamo/>.
- [7] 2025. OpenThoughts-114k. <https://huggingface.co/datasets/open-thoughts/OpenThoughts-114k>.
- [8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
- [9] Shaoyuan Chen, Yutong Lin, Mingxing Zhang, and Yongwei Wu. 2024. Efficient and Economic Large Language Model Inference with Attention Offloading. *arXiv preprint arXiv:2405.01814* (2024).

- [10] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiron Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhua Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, and S. S. Li. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR* abs/2501.12948 (2025). doi:10.48550/ARXIV.2501.12948 arXiv:2501.12948
- [11] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurélien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Rozière, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Graeme Nail, Grégoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel M. Kloumann, Ishan Misra, Ivan Evtimov, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelfer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, and et al. 2024. The Llama 3 Herd of Models. *CoRR* abs/2407.21783 (2024). doi:10.48550/ARXIV.2407.21783 arXiv:2407.21783
- [12] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC '24), Santa Clara, CA, USA, July 2024*. USENIX Association. <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>.
- [13] Jiaao He and Jidong Zhai. 2024. FastDecode: High-Throughput GPU-Efficient LLM Serving using Heterogeneous Pipelines. *CoRR* abs/2403.11421 (2024). doi:10.48550/ARXIV.2403.11421 arXiv:2403.11421
- [14] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de Las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Léo Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. *CoRR* abs/2401.04088 (2024). doi:10.48550/ARXIV.2401.04088 arXiv:2401.04088
- [15] Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2024. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *arXiv preprint arXiv:2411.01142* (2024).
- [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 611–626. doi:10.1145/3600006.3613165
- [17] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [18] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/43e9d647ccd3e4b7b5baab53f0368686-Abstract-Conference.html
- [19] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. 2025. Instinifer: In-storage attention offloading for cost-effective long-context llm inference. In *International Symposium on High-Performance Computer Architecture (HPCA)*.
- [20] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. [n. d.]. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA '24), Buenos Aires, Argentina, June 2024*. doi:10.1109/ISCA59077.2024.00019 <https://doi.org/10.1109/ISCA59077.2024.00019>.
- [21] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [22] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *CoRR* (2024). doi:10.48550/ARXIV.2407.00079 arXiv:2407.00079 <https://doi.org/10.48550/arXiv.2407.00079>.
- [23] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2407.00079* (2024).
- [24] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *CoRR*

- abs/2302.13971 (2023). doi:10.48550/ARXIV.2302.13971 arXiv:2302.13971
- [25] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023). doi:10.48550/ARXIV.2307.09288 arXiv:2307.09288
- [26] Tempest A. van Schaik and Brittany Pugh. 2024. A Field Guide to Automatic Evaluation of LLM-Generated Summaries. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14-18, 2024*, Grace Hui Yang, Hongning Wang, Sam Han, Claudia Hauff, Guido Zucco, and Yi Zhang (Eds.). ACM, 2832–2836. doi:10.1145/3626772.3661346
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*. 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [28] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh (Eds.). http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html
- [29] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yaqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. Qwen2 Technical Report. *CoRR* abs/2407.10671 (2024). doi:10.48550/ARXIV.2407.10671 arXiv:2407.10671
- [30] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, Alice Oh, Tristan Naumann, Amir Globerson, Kate Saenko, Moritz Hardt, and Sergey Levine (Eds.). http://papers.nips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html
- [31] Tianyang Zhong, Zhengliang Liu, Yi Pan, Yutong Zhang, Yifan Zhou, Shizhe Liang, Zihao Wu, Yanjun Lyu, Peng Shu, Xiaowei Yu, Chao Cao, Hanqi Jiang, Hanxu Chen, Yiwei Li, Junhao Chen, Huawen Hu, Yihen Liu, Huaqin Zhao, Shaochen Xu, Haixing Dai, Lin Zhao, Ruidong Zhang, Wei Zhao, Zhenyuan Yang, Jingyuan Chen, Peilong Wang, Wei Ruan, Hui Wang, Huan Zhao, Jing Zhang, Yiming Ren, Shihuan Qin, Tong Chen, Jiayi Li, Arif Hassan Zidan, Afrar Jahin, Minheng Chen, Sichen Xia, Jason Holmes, Yan Zhuang, Jiaqi Wang, Bochen Xu, Weiran Xia, Jichao Yu, Kaibo Tang, Yaxuan Yang, Bolun Sun, Tao Yang, Guoyu Lu, Xianqiao Wang, Lilong Chai, He Li, Jin Lu, Lichao Sun, Xin Zhang, Bao Ge, Xintao Hu, Lian Zhang, Hua Zhou, Lu Zhang, Shu Zhang, Ninghao Liu, Bei Jiang, Linglong Kong, Zhen Xiang, Yudan Ren, Jun Liu, Xi Jiang, Yu Bao, Wei Zhang, Xiang Li, Gang Li, Wei Liu, Dinggang Shen, Andrea Sikora, Xiaoming Zhai, Dajiang Zhu, and Tianming Liu. 2024. Evaluation of OpenAI o1: Opportunities and Challenges of AGI. *CoRR* abs/2409.18486 (2024). doi:10.48550/ARXIV.2409.18486 arXiv:2409.18486
- [32] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. [n. d.]. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), Santa Clara, CA, USA, July 2024*. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.